DEVOPS CI/CD ROLLBACK LAB

End-to-End Portfolio Demonstration & Interview Preparation

Target: DevOps Engineer interview at approximately 3 years of experience

Stack

GitHub • Jenkins • Python/Flask • Docker • Trivy • Kubernetes/Minikube • Helm

A practical guide built around the project actually demonstrated: successful normal deployment, simulated unhealthy release, Helm rollback, and application verification.

Prepared from the reported lab results and terminal outputs. Planned improvements are clearly separated from completed work.

1. How to use this guide

Use this as a learning and speaking guide, not as a script to memorize word-for-word. For a three-year DevOps interview, interviewers usually look for clear reasoning, command familiarity, incident ownership, trade-offs, and honest statements about what you personally implemented.

Learning order

- First understand the architecture and the purpose of each tool.
- Then rehearse the normal deployment path and the rollback path.
- Practise the commands until you can explain what each one verifies.
- Prepare the troubleshooting story and security remediation plan.
- Finally, rehearse the 7–10 minute demonstration and the Q&A.;

Accuracy rule

> This is a local personal lab. Do not describe it as a production system, claim business impact, or claim that all vulnerabilities are fixed. The successful normal build and the intentionally failed rollback-test build are different outcomes.

Repository
https://github.com/Anudeep1607/Automated-CI-CD-with-rollback

Project folder on the Mac used in the lab
~/Downloads/Projects/devops-cicd-rollback-macos-fixed/devops-cicd-rollback

2. Project summary and problem statement

Short explanation

I built a small Python web application and automated its path from source control to a local Kubernetes cluster. Jenkins checks out the code, runs unit tests, builds a Docker image, runs a Trivy scan, loads the image into Minikube, and deploys it with Helm. The pipeline includes a controlled parameter to simulate an unhealthy release and exercise rollback. I verified the recovered application using Kubernetes commands, Helm history, curl, and a browser.

Problem this lab demonstrates

Manual deployment steps are repeatable but error-prone. CI/CD makes the steps consistent; tests catch some regressions before deployment; containerization packages the app; Kubernetes runs the workload; Helm tracks releases; health checks help detect a failed rollout; rollback provides a recovery path.

What it does not prove

- It does not prove production-scale reliability or high availability.
- It does not prove the container is vulnerability-free.
- It does not establish an SLA or quantify business savings.
- It does not replace monitoring, alerting, change control, or production readiness reviews.

Interview framing: focus on the design, the evidence you gathered, the failures you debugged, and what you would improve next.

3. Architecture and request flow

```
Developer / code change
        |
        v
GitHub repository
  Jenkinsfile, app, tests, Dockerfile, Helm chart, rollback script
        |
        v
Jenkins Pipeline
  Checkout -> Unit Tests -> Docker Build -> Trivy Scan
        -> Load image into Minikube -> Helm deploy / verify / rollback
        |
        v
Minikube Kubernetes cluster
  Namespace: devops-demo
  Pod/Deployment: demo-app
  Service: demo-app (ClusterIP :5000)
        |
        v
kubectl port-forward 127.0.0.1:5000 -> service:5000
        |
        v
Browser / curl returns application JSON
```

Talk through the request

A browser request reaches the local port-forward process. kubectl forwards it to the Kubernetes Service. The Service selects the matching application pod by labels and forwards traffic to the target port. The Flask app responds with JSON. This lab uses a ClusterIP service, so it is not directly exposed outside the cluster.

Key distinction

A Pod is the running workload instance; a Service provides a stable in-cluster network endpoint; a Deployment manages desired replicas and rollout; Helm manages packaged Kubernetes resources and release revisions.

4. Tool-by-tool explanation

Git and GitHub — Git tracks source changes and snapshots them in commits. GitHub hosts the remote repository and enables collaboration and review.

Jenkins — executes the pipeline stages consistently. The job uses Pipeline script from SCM, so Jenkins reads the Jenkinsfile from the repository.

Python / Flask — the application responds with a small JSON payload. Unit tests provide an automated check before image build/deployment.

Docker — packages the application and its runtime dependencies into an image. A container is a running instance of an image.

Trivy — scans image contents for known vulnerabilities. A scan is evidence of findings at a point in time, not a guarantee of safety.

Kubernetes / Minikube — Kubernetes reconciles desired workload state. Minikube provides a local cluster for learning and demonstration.

Helm — packages and versions Kubernetes resources. `helm history` shows release revisions; `helm rollback` can restore a prior revision.

Port forwarding — creates a temporary local route to a Kubernetes Service. Keep the terminal process running while testing the URL.

5. Jenkins pipeline stages

1. Checkout

Jenkins retrieves the configured Git repository and branch. This makes the build reproducible against a source revision.

2. Unit Tests

The pipeline creates a Python virtual environment, installs the requirements, and runs unittest discovery. A test failure should normally stop later stages.

3. Build Container

Docker builds an image tagged with the Jenkins build number, for example `demo-app:3`. Build-specific tags help trace which build produced an image.

4. Security Scan

Trivy checks the image. In this lab, the scan is informational and does not block on findings. This is why a pipeline can finish successfully while reporting high-severity vulnerabilities.

5. Load Image into Minikube

The image is made available to the local cluster without requiring a remote image registry for this learning setup.

6. Deploy, Verify and Roll Back if Needed

Helm applies the release, Kubernetes rolls out the Deployment, and the script checks readiness/health. The simulated bad release exercises the recovery path.

> Before an interview, open the current Jenkinsfile and confirm exact stage names, commands, and parameter behavior. Explain only what the current code actually does.

6. Commands to know and explain

Tool versions

```
git --version
docker --version
kubectl version --client
minikube version
helm version
```

Cluster and workload

```
minikube status
kubectl get nodes
kubectl get pods -n devops-demo
kubectl get svc -n devops-demo
kubectl get endpoints demo-app -n devops-demo
kubectl describe pod <pod-name> -n devops-demo
kubectl logs <pod-name> -n devops-demo
```

Helm release

```
helm lint helm/demo-app
helm history demo-app -n devops-demo
helm status demo-app -n devops-demo
```

Application check

```
kubectl port-forward -n devops-demo svc/demo-app 5000:5000
# In another terminal:
curl -i http://127.0.0.1:5000/
```

Use `kubectl describe` and `kubectl logs` when the pod is not healthy. Avoid deleting resources first; inspect the failure and preserve evidence.

7. Git basics for someone new to Git

Working directory → staging area → local commit → remote push

```
git status
git diff -- README.md
git add README.md
git commit -m "Improve project README and rollback documentation"
git push origin main
```

What each command does

- `git status` shows changed, staged, and untracked files.
- `git diff` shows unstaged text changes.
- `git add` stages the selected file for the next commit.
- `git commit` saves a local snapshot with a message.
- `git push` uploads commits to the remote GitHub repository.

README update via GitHub website

- Open the repository and choose README.md; use the pencil/Edit button.
- Paste the reviewed README content and preview the Markdown.
- Commit with a meaningful message. Use a branch/pull request if that is your repo workflow.
- Refresh the repository page and verify the rendered README.

README update via Mac Terminal

```
cd ~/Downloads/Projects/devops-cicd-rollback-macos-fixed/devops-cicd-rollback
open -e README.md
# Edit and save, then:
git status
git diff -- README.md
git add README.md
git commit -m "Improve project README and rollback documentation"
git push origin main
```

If push is rejected, read the error and stop. Do not use force-push as a first fix. If Git says nothing to commit, check whether the editor saved the file.

8. Normal deployment demo

Before the interview

- Start Docker Desktop and confirm it is healthy.
- Start Minikube if needed and check `kubectl get nodes`.
- Open Jenkins and have the latest successful normal build ready.
- Keep screenshots/log excerpts available in case local services are slow.

Live demo sequence

```
cd ~/Downloads/Projects/devops-cicd-rollback-macos-fixed/devops-cicd-rollback
helm lint helm/demo-app
kubectl get pods -n devops-demo
kubectl get svc -n devops-demo
helm history demo-app -n devops-demo
kubectl port-forward -n devops-demo svc/demo-app 5000:5000
```

In another terminal run `curl -i http://127.0.0.1:5000/`, then show the browser response. Explain that the service is ClusterIP and port-forwarding is a temporary local test route.

Verified in this lab

A pod was observed `1/1 Running` with zero restarts. The service exposed port 5000/TCP. Curl returned HTTP 200 with JSON and the application reported version 4 after the rollback. The exact pod IP and age are transient and should not be memorized.

9. Rollback demo: what happened

The rollback test was intentionally triggered with `SIMULATE_BAD_RELEASE=true`. The simulated unhealthy release caused a Helm upgrade to fail because the Deployment was not ready. The pipeline then attempted recovery.

Observed Helm history

```
REVISION  STATUS      DESCRIPTION
1         superseded  Install complete
2         superseded  Upgrade complete
3         failed      Upgrade failed; Deployment not ready
4         deployed    Rollback to 2
```

Interpretation

- Revision 3 records the failed upgrade attempt.
- Revision 4 is the rollback action and is marked deployed.
- The rollback description says it restored revision 2.
- Afterward, the pod was Running and curl/browser returned HTTP 200.

Do not confuse these two version numbers

Helm revision 4 means the fourth Helm release revision, while the JSON `version` field is the application-reported version. They are not the same numbering system.

Why Jenkins showed FAILURE

The test intentionally created a failed deployment. The recovery step succeeded, but the pipeline can still mark the overall test build as failed to preserve the failed deployment outcome. State both facts clearly: deployment failed; recovery succeeded.

10. Troubleshooting story: Jenkins syntax

Symptom

The first Jenkins run fetched the Jenkinsfile but failed Groovy compilation because `PATH+HOMEBREW` was used as an environment variable assignment inside the declarative environment block.

Reasoning

The failure happened before the unit tests or Docker build. That points to pipeline parsing/configuration rather than an application runtime issue.

Resolution

The environment entry was corrected to a supported `PATH` assignment, committed and pushed. Subsequent builds advanced beyond the compilation error.

Interview answer

“I first classified the failure by stage. Jenkins failed while compiling the Pipeline, before any workload stage ran. I checked the Jenkinsfile environment block, corrected the invalid assignment syntax, committed the fix, and reran the job. I then used the next error output rather than assuming the whole pipeline was fixed.”

Lesson

Separate syntax/configuration failures from test failures and deployment failures. Fix the earliest failing layer first, then rerun and inspect the next stage.

11. Troubleshooting story: malformed requirements

Symptom

Pip reported an invalid requirement resembling `Flask>=3.0,<4.0\n`. The newline escape had been written as literal text instead of a real line ending.

Resolution used

```
printf 'Flask>=3.0,<4.0\n' > app/requirements.txt
cat app/requirements.txt
git add app/requirements.txt
git commit -m "Fix Flask requirements file"
git push
```

This command was appropriate for the lab's single dependency. It overwrites the target file, so it should not be blindly reused in a project with additional dependencies.

Interview answer

“The test stage failed before running the tests because pip could not parse the dependency file. I inspected the file contents, corrected the line ending using printf, verified the content, committed the change, and reran Jenkins. The tests then passed.”

Lesson

When an error points to a config/dependency file, inspect the literal file contents before changing application code. Validate the smallest possible fix.

12. Troubleshooting story: browser 403

Symptom

A regular browser tab displayed HTTP 403 at `127.0.0.1:5000`, although the port-forward process started.

Evidence collected

```
curl -i http://127.0.0.1:5000/
kubectl get endpoints demo-app -n devops-demo
kubectl get svc demo-app -n devops-demo -o yaml
```

Curl returned HTTP 200 and valid JSON; the service endpoint existed at the time of the check. The user then opened the URL in Chrome Incognito and received the expected JSON.

What can be concluded

The service and application were responding correctly through curl, and Incognito browser access worked. The exact root cause of the original browser-only 403 was not proven.

Interview answer

“I did not assume the Kubernetes deployment was broken based on the browser message alone. I tested the endpoint with curl, checked the service endpoint and port mapping, and then tested in Incognito. Curl returned 200 and Incognito worked, so I recorded the initial 403 as a browser/session-specific symptom with root cause not confirmed.”

This is a strong troubleshooting habit: distinguish confirmed facts from a hypothesis.

13. Security findings and remediation plan

Observed Trivy summary

The displayed scan reported 44 high-severity OS-package findings and 4 high-severity Python-package findings, with zero critical findings in that summary. The scan was configured as informational, so findings did not block the build.

Correct statement

> “The scan executed and identified high-severity vulnerabilities. The current pipeline does not fail on those findings. I have documented remediation as follow-up work; I do not claim the image is vulnerability-free.”

Planned remediation — not yet verified complete

- Export the full scan output and identify CVE, package, installed version, fixed version, and whether the issue is fixable.
- Prioritize fixable high-severity findings and assess the package's use and exposure.
- Update the base image and compatible Python dependencies; pin versions where appropriate.
- Rebuild and rerun unit tests; rescan the final image.
- Compare before/after findings and document residual items that have no available fix.
- Set a deliberate CI policy for critical/high thresholds, with documented exception handling.
- Consider generating an SBOM and retaining scan artifacts.

Why not just fail all builds immediately?

A team should agree on severity thresholds, fixability, exceptions, and rollout policy. A hard gate can be appropriate, but it should be intentional and should not hide unresolved risk.

Do not say remediation is complete until a new scan proves the findings were reduced or resolved.

14. Interview Q&A;: CI/CD, Git and Jenkins

Q: What is CI/CD?

CI continuously integrates changes with automated validation. CD can mean continuous delivery (ready to release) or continuous deployment (automatically released). This lab automates tests, image build, scan, deployment, and verification.

Q: Why use a Jenkinsfile?

It stores pipeline-as-code alongside the project, making stages reviewable and version-controlled.

Q: What is the difference between a Git commit and push?

Commit records a snapshot in the local repository; push uploads committed history to a remote repository such as GitHub.

Q: Why use build-specific image tags?

They help trace a deployed image back to a pipeline build. Mutable tags such as `latest` are less precise for auditing and rollback.

Q: Why did the security scan not fail the build?

The lab config explicitly made Trivy informational. That behavior must be disclosed; future work is to define a policy threshold and remediation workflow.

Q: What do you do when a Jenkins build fails?

Open Console Output, identify the first failing stage and exact error, determine whether it is syntax, dependency, build, registry/cluster, or rollout related, reproduce safely, make a minimal fix, rerun, and capture evidence.

15. Interview Q&A;: Docker and Kubernetes

Q: Image vs container?

An image is a packaged template; a container is a running instance created from it.

Q: Pod vs Deployment?

A Pod is the smallest deployable workload unit. A Deployment manages ReplicaSets and desired pod replicas and supports rolling updates.

Q: What is a Service?

A Service provides a stable network abstraction over selected Pods. In this lab, `demo-app` is ClusterIP and port 5000.

Q: What does `1/1 Running` tell you?

The pod reports one of one containers ready and the pod phase is Running. It is a useful signal, but it does not alone prove every application function works; an HTTP health check adds evidence.

Q: What is Minikube?

A tool for running a local Kubernetes cluster, useful for development and learning without a managed cloud cluster.

Q: What would you check if a pod is Pending or CrashLoopBackOff?

```
kubectl get pods -n devops-demo
kubectl describe pod <pod-name> -n devops-demo
kubectl logs <pod-name> -n devops-demo
kubectl get events -n devops-demo --sort-by=.lastTimestamp
```

Check scheduling/resource constraints, image pull errors, configuration, probes, process exit codes, and recent events before making changes.

16. Interview Q&A;: Helm and rollback

Q: What is Helm?

Helm packages Kubernetes manifests into charts and manages named releases and their revisions.

Q: Chart vs release?

A chart is the package/template set; a release is an installed instance of that chart in a cluster.

Q: How did you prove rollback worked?

I inspected `helm history`, saw the failed upgrade revision and a later deployed rollback revision, confirmed the pod was Running, and checked that curl/browser returned HTTP 200.

Q: Why can Jenkins show FAILURE if recovery succeeded?

The test deliberately attempted a failed deployment. The pipeline can retain a failed overall status to report that the attempted release was unhealthy, even if the rollback stage restored service. The deployment result and recovery result are separate.

Q: What are rollback limitations?

Rollback of Kubernetes manifests does not automatically reverse irreversible database migrations or external side effects. Production rollback needs backward-compatible changes, health checks, observability, and a data recovery strategy.

Q: What is a readiness probe?

It signals whether a container is ready to receive traffic. A liveness probe helps detect a container that should be restarted. This project’s exact probe configuration should be checked in the Helm templates before claiming specific probe behavior.

17. Interview Q&A;: troubleshooting and security

Q: How do you distinguish a Service problem from an app problem?

Check pod readiness and logs, Service selectors and ports, EndpointSlices/endpoints, then test from inside the cluster or use port-forward and curl. Compare network-level response with browser behavior.

Q: What is a vulnerability scan?

A scanner compares package and OS metadata against vulnerability databases. Results depend on database freshness, package identification, fix availability, and configuration.

Q: Does a green pipeline mean the image is secure?

No. A pipeline can be green if the scan is informational or configured not to fail. In this lab, Trivy reported high findings, so I would disclose them and track remediation.

Q: How would you improve secrets management?

Use Jenkins Credentials or a dedicated secret manager, apply least privilege, rotate secrets, mask sensitive output, and avoid committing secrets to Git.

Q: How would you improve this project for production?

- Use a registry with immutable image tags and artifact provenance.
- Add resource requests/limits and review security context and probes.
- Introduce a documented vulnerability gate and SBOM retention.
- Use branch protection and review, secrets management, notifications, and logs/metrics.
- Test in a representative non-production cluster and plan for database compatibility.

18. Interview demo script: 7–10 minutes

0:00–1:00 — Introduce the goal

“This is a personal DevOps lab that automates testing, image build, security scanning, Kubernetes deployment, and rollback using Jenkins and Helm.”

1:00–2:00 — Show the repository

Show the Jenkinsfile, app, tests, Dockerfile, Helm chart, and rollback script. Explain each file's role without reading code line-by-line.

2:00–4:00 — Show normal pipeline evidence

Open the successful normal Jenkins build and explain checkout, tests, Docker build, Trivy, image load, Helm deploy, and health check.

4:00–5:00 — Verify the running app

```
kubectl get pods -n devops-demo
kubectl get svc -n devops-demo
curl -i http://127.0.0.1:5000/
```

5:00–7:00 — Show rollback evidence

```
helm history demo-app -n devops-demo
# Show saved Jenkins console log for SIMULATE_BAD_RELEASE=true
```

7:00–8:00 — Explain security findings

Disclose the 44 OS and 4 Python high-severity findings reported in the scan summary. Explain that remediation is planned and not yet verified.

8:00–10:00 — Reflect

Explain one troubleshooting example, one limitation, and the next improvement. Do not trigger a new unhealthy release live if the environment is unstable or time is short.

19. Resume wording and project talking points

Suggested resume title

Automated CI/CD Pipeline with Kubernetes Deployment and Helm Rollback | Personal DevOps Project

Resume bullets

- Built a Jenkins CI/CD pipeline to run Python unit tests, build Docker images, scan images with Trivy, and deploy a Flask application to Kubernetes using Helm.
- Demonstrated an unhealthy-release scenario and verified Helm rollback to a previous healthy release using release history, pod status, and HTTP response checks.
- Troubleshot Jenkins Pipeline syntax and malformed Python dependency configuration, restoring pipeline execution and documenting root causes and fixes.
- Identified high-severity OS and Python package findings from Trivy and documented a remediation plan rather than treating pipeline success as proof of a vulnerability-free image.

Keep claims defensible

- Label this as a personal project, not production experience.
- Do not claim zero vulnerabilities or completed remediation.
- Do not invent uptime, time savings, scale, number of users, or business impact.
- Only mention technologies and commands you can explain during follow-up questions.

30-second project pitch

“I created a Jenkins pipeline for a Python service that tests the code, builds and scans a Docker image, loads it into Minikube, and deploys through Helm. I then simulated an unhealthy release and verified that Helm recorded the failed revision and a successful rollback. I also debugged pipeline syntax and dependency-file errors, and documented the remaining Trivy findings as remediation work.”

20. Final checklist and next steps

Before publishing the README

- Check that README commands and file names match the current GitHub repository.
- Remove secrets, personal tokens, local credentials, and private infrastructure details.
- Confirm the normal build and rollback-test build are described separately.
- Keep the security findings and remediation status accurate.
- Commit the README with a clear message and verify it renders on GitHub.

Before the interview

- Explain every stage in the Jenkinsfile.
- Know the difference between Docker image tags and Helm revisions.
- Run the verification commands without copying blindly.
- Practise the failure → diagnosis → fix → verification story.
- Have screenshots ready as a fallback for local environment issues.

Follow-up remediation work

- Investigate and remediate the reported Trivy high-severity findings, then rescan.
- Consider SBOM generation and a documented severity gate.
- Review Dockerfile hardening, Kubernetes resource settings, probes, and security context.
- Improve README with architecture diagram and sanitized evidence screenshots.

Final takeaway

> A strong interview story is not “everything worked perfectly.” It is “I built the workflow, investigated failures systematically, proved recovery with evidence, and clearly identified the remaining risks.”